COMP 418/518: Homework 2

[Weight of homework: 25% of the final grade]

Authors (fill in your names and NetIDs here)

Due on February 24, 2026
(released on February 6, 2026)

Instructions: For this homework, you should work in groups of 3–4 people (i.e., the groups that have been declared and accepted). Only one member of the group should submit on Gradescope. All group members must be included in the submission on the Gradescope system (otherwise no credit will be given). Also specify in your submission file all the members of your group (name and NetID) using the `\authors` macro. The submission must be typed in LaTeX using the provided template. The use of AI systems (including, but not limited to, ChatGPT, Gemini, Copilot, and Claude) is entirely allowed. No discussion or collaboration is allowed outside of the group. Late submissions are not accepted.

You should provide *pseudocode* for all the algorithms that you describe.

Note: We will request that you upload the LaTeX source for your submission. More information regarding how to make the source submission will be made available in a few days.

Grading: The homework will be graded based on correctness, completeness, and the quality of the provided solutions.

Problem 1 [20 points]

Many IoT applications involve the processing of signals that are generated by sensors. Processing these signals often involves *filtering*, a signal processing technique used to remove noise from the signal. The moving (sliding) average that we have seen in class is an example of an *FIR (Finite Impulse Response)* filter that removes high-frequency noise.

You are asked to design a streaming algorithm for another FIR filter, which is a sliding-window computation. The size of the window is W. Assume that the input stream consists of real numbers. At every step (after the first W stream elements have been received), the filter should output the quantity

$$x_1 + 2x_2 + 3x_3 + \cdots + Wx_W,$$

where $x_1, \ldots, x_W$ are the last W elements of the stream (i.e., the current window with x_W being the most recent element).

Describe the most space- and time-efficient streaming algorithm that you can think of. Explain why your algorithm is correct. Give a precise description of the resources (space and time per item) that your algorithm uses.

Solution

Fill me in.

Problem 2 [30 points]

Suppose that the input stream is a sequence of measurements from a smart IoT sensor. The measurements are integers, i.e., elements of $\mathbb{Z} = \{\ldots, -2, -1, 0, 1, 2, \ldots\}$. We use the notation $x[n]$, where $n \in \mathbb{N}$, for the input stream. We also call x the *input signal*. The value $x[n]$ is the sensor measurement at time instant n.

We want to perform a sliding-window computation, where $W \geq 2$ is the size of the window. The window at a time instant $n \in \mathbb{N}$ contains the measurements

$$x[\max(0, n - W + 1)], \ldots, x[n - 1], x[n] \ .$$

For example, if $W = 3$, then the window at time instant $n = 5$ contains the measurements $x[3], x[4], x[5]$. We use the notation $y[n]$, where $n \in \mathbb{N}$, for the output stream we want to compute, i.e., the *output signal*. The value $y[n]$ is the output at time instant n. Let $T_1, T_2 \in \mathbb{Z}$ be two *threshold* values that satisfy $T_1 < T_2$. Define the output value

$$y[n] = \begin{cases} 1, & x[i] \in [T_1, T_2) \text{ and } x[j] \in [T_2, \infty) \text{ for some } i, j \in \{\max(0, n - W + 1), \ldots, n\} \text{ with } i < j \\ 0, & \text{otherwise} \end{cases}$$

for every time instant $n \in \mathbb{N}$. In other words, $y[n] = 1$ iff the window (at time instant n) contains a measurement in the interval $[T_1, T_2) = \{v \mid T_1 \leq v < T_2\}$ followed by a measurement in the interval $[T_2, \infty) = \{v \mid v \geq T_2\}$.

Describe the most space- and time-efficient streaming algorithm that you can think of for this sliding-window computation. Explain why your algorithm is correct. Give a precise description of the resources (space and time per item) that your algorithm uses.

Solution

Fill me in.

Problem 3 [15 points]

Recall from the lectures the "sliding max" problem, that is, the streaming problem of calculating the maximum over a sliding window.

Give an algorithm for the sliding max problem with worst-case time per item $O(\log W)$, where W is the size of the window. Explain why your algorithm is correct. What is the space and total time complexity of your algorithm?

Solution

Fill me in.

Problem 4 [40 points]

Suppose that the input items are drawn from the set $\{0, 1, \ldots, R-1\}$, where R is a (small) *constant*.

(1) Give a space-optimal (asymptotically) algorithm for the sliding max problem (the window size is W). Measure space (memory) in bits.

(2) Prove a space lower bound (for the problem) that matches the upper bound for your algorithm.

Solution

Fill me in.

Problem 5 [20 points]

An ***IoT device A*** generates *messages* that are nested lists/sets of (arbitrarily large) natural numbers. We will define now the set D, which should be thought of as an abstract mathematical representation of such messages. Formally, D is the smallest set that satisfies the following properties (which are implicitly universally quantified):

1. [Number] If $n \in \mathbb{N}$ then $n \in D$ ("every natural number is a message").
2. [List] If $x_1, x_2, \ldots, x_n \in D$ then $[x_1, x_2, \ldots, x_n] \in D$.
3. [Set] If $x_1, x_2, \ldots, x_n \in D$ then $\{x_1, x_2, \ldots, x_n\} \in D$.

Empty lists $[\,]$ and empty sets $\{\}$ are messages. For example, $[4, \{2, 3\}, [1, 2, \{\}], [6, [\,]], 7]$ is a message. The messages are transmitted in serialized form, where numbers are represented as sequences of decimal digits. A *serialized message* is a sequence over the following alphabet Σ of symbols:

`0 1 2 3 4 5 6 7 8 9 [ ] { } ,`

It is also allowed to have (any number of) space symbols separating numbers, commas, brackets, and braces. For example, this is a serialized message (observe the presence of space symbols):

`[4, {2, 3}, [1, 2, {}], [6, []], 7]`

The ***IoT device B*** receives a stream over Σ (***message stream***), which should be a sequence of serialized messages that are separated by (any number of) space symbols. For example,

`[ ] 100 {1} [2, 3] [4, {7,8}, 5] { }`

is a stream that consists of 6 complete serialized messages.

Describe a streaming algorithm for Device B that processes an input message stream. As soon as the algorithm sees a complete (serialized) message, it should output the total number of messages processed so far. This means that the algorithm should not produce output at every step. As soon as the algorithm notices that a message is *malformed* (e.g., `[1, 2}` or `[1, [2, 3]]]` or `[1, 2,]`), it should start emitting the error value -1 at every step.

Explain why your algorithm is correct. Give a precise description of the resources (space and time per item) that your algorithm uses.

Solution

Algorithm Overview

We use a **finite-state automaton** combined with a **stack** S to track bracket/brace nesting. The automaton has five normal states plus an error state:

- `TOP`: Between top-level messages. Expecting the start of a new message or a space.
- `NUM`: Reading digits of a number.
- `OPEN`: Just opened a bracket/brace. Expecting an element or a matching close.
- `SEP`: Just saw a comma. Expecting the next element.
- `ELEM`: Just finished an element inside a list/set. Expecting a comma or a matching close.
- `ERR`: Malformed input detected. Emit -1 at every subsequent step.

We also maintain a counter C (number of complete messages).

Pseudocode

`INIT()`:

- state $\leftarrow$ `TOP`, $S \leftarrow$ empty stack, $C \leftarrow 0$

`NEXT(`σ`)`: *// Process incoming symbol* $\sigma \in \Sigma$

1. **If** state = `ERR`: output -1 and return.
2. **If σ is a space:**
   - If state = `NUM` and S is empty: $C \leftarrow C + 1$; state $\leftarrow$ `TOP`; output C; return.
   - If state = `NUM` and S is non-empty: state $\leftarrow$ `ELEM`; no output; return.
   - If state $\in \{$`TOP`, `OPEN`, `SEP`, `ELEM`$\}$: no change; no output; return.
3. **If σ is a digit:**
   - If state $\in \{$`TOP`, `OPEN`, `SEP`$\}$: state $\leftarrow$ `NUM`; no output; return.
   - If state = `NUM`: stay in `NUM`; no output; return.
   - If state = `ELEM`: state $\leftarrow$ `ERR`. *// Adjacent values, no comma.*
4. **If** $\sigma \in \{$`[`, `{`$\}$ (open bracket/brace):
   - If state $\in \{$`TOP`, `OPEN`, `SEP`$\}$: push σ onto S; state $\leftarrow$ `OPEN`; no output; return.
   - Otherwise: state $\leftarrow$ `ERR`. *// e.g., `123[` or `][` with no space.*
5. **If** $\sigma \in \{$`]`, `}`$\}$ (close bracket/brace):
   - If state = `SEP`: state $\leftarrow$ `ERR`; return. *// Trailing comma: `[1,]`*
   - If state $\in \{$`NUM`, `OPEN`, `ELEM`$\}$ and S is non-empty:
     - Let $o = S$.top(). If ($o =$ `[` and $\sigma =$ `]`) or ($o =$ { and $\sigma =$ }): pop S.
     - Otherwise: state $\leftarrow$ `ERR`; return. *// Mismatch: `[1, 2`}*
     - If S is now empty: $C \leftarrow C + 1$; state $\leftarrow$ `TOP`; output C; return.
     - Else: state $\leftarrow$ `ELEM`; no output; return.
   - Otherwise: state $\leftarrow$ `ERR`. *// e.g., `]` at top level.*
6. **If $\sigma =$ `,` (comma):**
   - If state $\in \{$`NUM`, `ELEM`$\}$ and S is non-empty: state $\leftarrow$ `SEP`; no output; return.
   - Otherwise: state $\leftarrow$ `ERR`. *// Comma at top level, double comma, etc.*

Correctness

The automaton enforces the grammar of valid messages:

- A message is either a number (sequence of digits), a list `[...]`, or a set {...}.
- List/set elements are separated by commas. Empty lists `[]` and sets { } are allowed (via the `OPEN` $\rightarrow$ close transition).
- The stack S ensures correct nesting and matching of brackets/braces.
- Transitions to `ERR` capture all malformed cases: mismatched brackets (`[1,2`}), extra closing brackets (`[1,[2,3]]]`), trailing commas (`[1,2,]`), missing commas, commas at top level, and double commas.
- Once in `ERR`, the algorithm emits -1 at every subsequent step.

Example trace. For the stream "`[ ] 100 {1}`", the algorithm transitions: `TOP` $\xrightarrow{[}$ `OPEN` $\xrightarrow{\sqcup}$ `OPEN` $\xrightarrow{]}$ `TOP` (output $C{=}1$) $\xrightarrow{\sqcup}$ `TOP` $\xrightarrow{1}$ `NUM` $\xrightarrow{0}$ `NUM` $\xrightarrow{0}$ `NUM` $\xrightarrow{\sqcup}$ `TOP` (output $C{=}2$) $\xrightarrow{\{}$ `OPEN` $\xrightarrow{1}$ `NUM` $\xrightarrow{\}}$ `TOP` (output $C{=}3$).

Resource Analysis

- **Time per item:** $O(1)$. Each symbol requires a constant number of operations (check state, possibly push/pop the stack, update counter).
- **Space:** $O(d_{\max} + \log n)$ bits, where $d_{\max}$ is the maximum nesting depth and n is the stream length. The stack uses $d_{\max}$ bits (1 bit per level, distinguishing `[` from {). The counter C uses $O(\log n)$ bits. The automaton state uses $O(1)$ bits (one of 6 states = 3 bits). In the worst case, $d_{\max} = O(n)$ (e.g., `[[[...]]]`), so the space is $O(n)$ bits.

Problem 6 [125 points]

Implement the approximation algorithm for counting 1's over a sliding window using the provided code files. Include the benchmarking results and any other relevant information below.

- Zip up the entire folder with your solution (only the source code!) and submit it on Gradescope.
- Your implementation should only call malloc ***once*** (during the initialization of the state). After this call of malloc during initialization, there should be no other dynamic memory allocation. The purpose of this restriction is to be faithful to an embedded IoT device that has a small amount of memory and offers no dynamic memory allocation.
- All the data structures used by your algorithm should be implemented from scratch.

Solution

Algorithm Overview

We implement the DGIM (Datar–Gionis–Indyk–Motwani) approximation algorithm for counting 1's over a sliding window of size W. The algorithm maintains a collection of *buckets*, where each bucket stores:

- A **timestamp** (the time of the most recent 1 it represents), and
- A **power** p (the bucket represents 2^p ones).

Setting the parameter k determines the accuracy: we allow at most $r+1 = k+1$ buckets per power level.

Data Structure

We use **per-power-level ring buffers** (circular queues), each storing only timestamps for buckets of that power. Specifically:

- For each power level $p \in \{0, 1, \ldots, \lfloor \log_2 W \rfloor + 2\}$, we maintain a separate ring buffer of capacity $r + 2$.
- All ring buffers are allocated from a **single contiguous block** via one call to `malloc`. The total memory is $(r + 2) \times$ (number of power levels) $\times$ sizeof(`uint32_t`) bytes.
- We also maintain a running `total_sum` of all bucket sizes and a `highest_power` tracker.

This design achieves $O(1)$ amortized time per merge (just pop from one ring and push to another), and avoids any additional dynamic memory allocation after initialization.

Processing Each Item

On each incoming bit:

1. **Expire**: Remove the oldest bucket (at the tail of the highest occupied power level) if its timestamp is outside the window.
2. **Insert**: If the bit is 1, push a new entry (timestamp, power 0) onto ring 0.

3. **Merge cascade**: Starting from power $p = 0$, if the ring for power p has $r+2$ entries, pop the two oldest, and push one merged entry (with the more recent of the two timestamps) onto the ring for power $p+1$. Repeat for $p+1$ if needed.

4. **Query**: Return $\texttt{total_sum} - 2^{p_{\max}} + 1$, where $p_{\max}$ is the power of the oldest bucket.

Estimate and Error Analysis

The estimate is:

$$\hat{C} = \texttt{total_sum} - 2^{p_{\text{oldest}}} + 1$$

where p_{oldest} is the power of the oldest (highest-power) bucket.

No overestimation: The oldest bucket has not been expired, so its timestamp (the most recent 1 it represents) is inside the window. Thus at least 1 of its $2^{p_{\text{oldest}}}$ constituent 1's is inside the window. Therefore:

$$\hat{C} = \texttt{total_sum} - 2^{p_{\text{oldest}}} + 1 \leq \text{true count}$$

Error bound: The error is at most $2^{p_{\text{oldest}}} - 1$. With $r = k$ buckets per power level, the minimum total sum when the oldest bucket has power p is:

$$\texttt{total_sum} \geq k \cdot (2^0 + 2^1 + \cdots + 2^{p-1}) + 2^p = k(2^p - 1) + 2^p$$

Thus the relative error satisfies:

$$\frac{\text{error}}{\text{true count}} \leq \frac{2^p - 1}{k(2^p - 1) + 1} < \frac{1}{k}$$

Complexity

- **Space**: $O(k \cdot \log W)$ timestamps $= O(k \log W)$ words. A single `malloc` call allocates all memory at initialization.

- **Time per item**: $O(1)$ amortized. Each item causes at most one insertion and one expiry, both $O(1)$. Merges are amortized $O(1)$ since each bucket is merged at most $O(\log W)$ times over its lifetime.

Benchmarking Results

We benchmarked the exact and approximate algorithms over a stream of 50,000,000 items (all 1's), across window sizes $W \in \{10, 100, 1000, 10{,}000, 100{,}000, 1{,}000{,}000\}$ and accuracy parameters $k \in \{10, 100, 1000\}$. Each configuration was run for 3 trials.

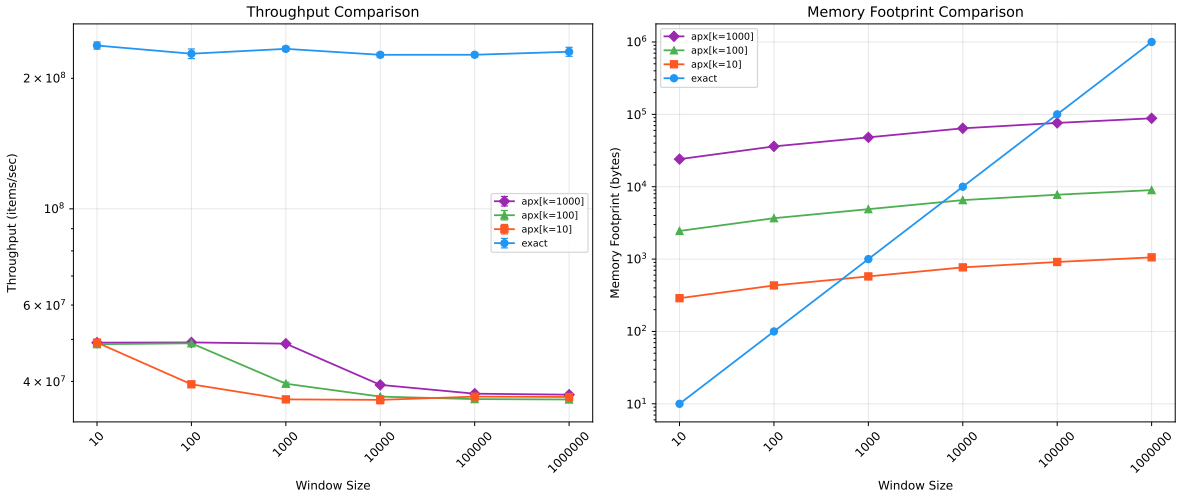

Figure 1: Throughput comparison (items/sec, log scale). The exact algorithm achieves ∼230M items/sec regardless of W. The approximate algorithm achieves ∼37–49M items/sec (compiled with `-O0`, no optimizations). Throughput is relatively insensitive to k and W.

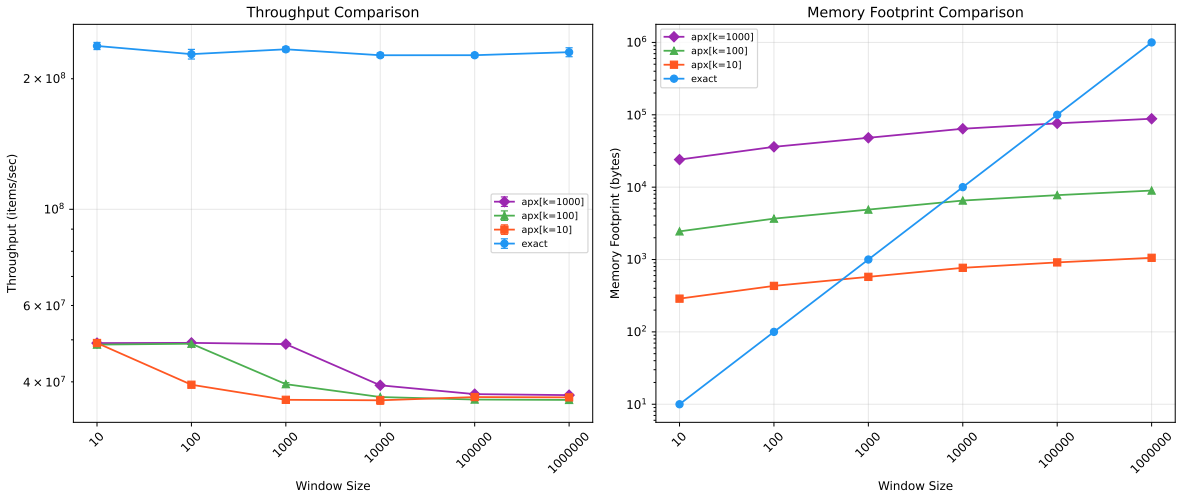

Figure 2: Memory footprint comparison (bytes, log scale). The exact algorithm uses W bytes ($\Theta(W)$). The approximate algorithm uses $O(k \log W)$ bytes — for example, at $W = 1{,}000{,}000$ with $k = 10$, it uses only 1,056 bytes vs. 1,000,000 bytes for exact, a ∼950× reduction.